

AI enabled SDLC:

We have come far, we have a long way to go

Jaideep Khandelwal

\$ whoami

- **Founder & CTO at One2N.** 🧑💻
- **Reliability, Data and Backend**
Product Engineering. 🔧
- **15+** years with startups, SMEs. ⌚
- From **Buzzword** driven development
to **impact** driven development. 📈
- Retro Games 🎮 🎮
- Building **Lego** sets 🧱



Vibe Coding

Usual workflow

- **Write a prompt**
- **Receive generic code that does not make sense**
- **You correct it with more context**
- **Better but still misaligned**
- **You Give up (the AI does not..)**

What your AI agent looks like

- **Infinitely fast**
- **Technically excellent - in general**
- **Knows nothing specific about your project**
- **No memory of yesterday's session**
- **Patterns from other codebases that don't fit yours**



Nishant | Citrea   @nishant_zk · 20h



Ek Developer Vibe Coder ban sakta hai !
But
Ek Vibe Coder kabhi developer nahi ban sakta !



 105

 300

 3.7K

 85K





Post



Uncle Bob Martin ✓

@unclebobmartin



Vibe coding is not the same as disciplined agentic development.

9:16 PM · Mar 10, 2026 · **55K** Views

AI Enabled SDLC

Habit changes

- **Brief your AI before you start**
- **Design before you code**
- **Write your team's rules**
- **Keep a decision log (Memory)**
- **Capture what you learn**

Brief your AI before your start

- What and What NOT
- Define stack, framework
- Non-negotiables
- Project Structure
- Code Patterns
- Naming Conventions
- <Add custom instruction>

Gotchas

- Outdated CLAUDE.md files can mislead.
- Brevity wins.
- Institutional memory lives in the repo.
- AI-generated first drafts need human review.

Design before you code

- **Scope :** What does this actually do? What's explicitly out of scope for now?
- **Components:** What are the components?
- **Data flow :** How do they connect? What happens on the error paths?
- **Interfaces:** What do the function signatures and types look like?
- **Execution:** Execute the discussion post approval

Gotchas

- **Explicitly gate each step. Explicitly say "wait for approval at each step".**
- **Push back during design. Review carefully.**
- **Write approved decisions into the implementation-guide.**
- **Be careful about what is in the scope and out of scope.**

Write your team's rules

- **Scenario:** Imagine, two developers using the same AI tool
- **Problem:** Different output quality. The senior engineers thinking is in the head
- **Fix** is to write instructions.
 - **Generation instructions:** How to build new code , patterns, naming, error handling
 - **Refactoring instructions:** How to improve existing code without breaking contracts
 - **Security checklist:** Your specific threat model
 - **Review checklist:** Your quality gate, what gets blocked, what gets flagged

Gotchas

- Skills that try to cover everything enforce nothing.
- Skill files go stale too. Keep it updated.
- The first version could be wrong. Review them.

Keep a decision log (Memory)

- **Problem:** AI memory is ephemeral
- **Problem:** Waste time re-explaining context at the start of session
- **Fix** is to create living decision document - a working record
 - Decision record not just for project but also features as well
 - Implementation Guide - Intention: What to build and why
 - Execution Report - Result: What was built, what changed, how to run it

Gotchas

- Document rejected alternatives, not just the happy path.
- Be careful about what to document per story. Can end up with multiple files.
- Write the execution report immediately after implementation.

Capture what you learn

- **Problem 1:** Repeated mistakes collaborating with AI
- **Problem 2:** No knowledge base from mistakes
- **Fix** is to create living decision document - a working record
- **After each story. Three questions:**
 - What did AI consistently get wrong?
 - Add a guardrail to **CLAUDE.md** or a skill file
 - What prompt worked really well?
 - Save it as a template in **context/04-prompt-templates.md**
 - What was missing from the briefing that I had to explain mid-session?
 - Add it to **context/01-base-instructions.md**

Gotchas

- **Assign retro ownership explicitly.**
- **Guardrails belong in a PR, not a chat message.**
- **When a session goes well, make a note of what prompt worked.**

Onboarding to existing code

- **Treat AI agent like an engineer who joins your team**
- **Produce**
 - **base_instructions.md - Stack, Rules, Non-negotiables**
 - **file_patterns.md - Package Layout, Naming conventions, Code Patterns**

Questions?

Twitter - @jdk2588

LinkedIn - [linkedin.com/in/jdk2588](https://www.linkedin.com/in/jdk2588)

